



ONE TAP CINEMA

ITI Final Project

A cross-platform Flutter movie discovery & watchlist application

By Malak Ahmed Mohamed Abd ElRahim

Table of Contents

Project Overview	4
Features	4
Splash Screen:	4
Firebase Authentication:.....	4
Home Feed:	4
Password change:.....	4
Search function:	4
Browse By Genre:.....	4
Movie Details:	4
Favorites and Watch lists:.....	4
Profile screen:.....	4
Dark/Light Mode:.....	4
Content Filtering:	5
Trigger Warning/ Parent Guide:	5
Technologies Used:	5
Framework:	5
State Management.....	5
Network and Api:	5
Database:.....	5
Firebase	5
Media & UI	5
Dev Tools	5
Architecture:	5
Data Flow:	7
Installtion Guide:.....	7
Steps:	7
API setup Instructions:.....	7
Code details:.....	10
FireBase:	10
Code details:.....	11
DataBase:	11
Application Flow:	11
Known Limitations:	12
AppTheme:	12
Check Notion file For Editing history:	13
ScreenShots:	14
Old Screenshots:	14

New and Final Screen Shots:	14
--	-----------

Project Overview

One Tap Cinema is a cross-platform Flutter application that allows users to discover movies, explore detailed information and manage personal watchlists. It fetches movie data from TMDB API. It uses Firebase Authentication to handle Login and Sign up. It stores watchlist data locally using SQLite for mobile and Shared Preferences for Web. It has been tested on Andriod and Web and assumed to be working well for the rest of devices.

Features

Splash Screen:

- Contains the TMDB logo and an AI generated animation video of a theatre curtain in ghibli style that matches the color of the app

Firebase Authentication:

- Contains an Email/password Authentication with real-time state listening

Home Feed:

- Displays Popular, Now Playing, and Top Rated movies that gets updated from TMDB regularly.

Password change:

- Can change your password

Search function:

- You can search using movie names or keywords.
- Paginated results to avoid errors

Browse By Genre:

- Browse movies filtered by genre with paginated results.

Movie Details:

- Contains Cast, tagline, genres, storyline, runtime and others.

Favorites and Watch lists:

- Lists are stored locally on your device.

Profile screen:

- Display user info and quick links to watchlists.

Dark/Light Mode:

- Full theme toggle with main Theatre colors (Red and Gold)

Content Filtering:

- Adult and Softcore content is automatically filtered from all movie results

Trigger Warning/ Parent Guide:

- Launches a google search with the movie name and content warnings.

Technologies Used:

Framework:

- Flutter and Dart

State Management

- Provider ^6.1.2

Network and Api:

- http ^1.2.2
- url_launcher ^6.3.1

Database:

- sqflite ^2.3.3
- shared_preferences ^2.5.5
- path ^1.9.0

Firebase

- firebase_core ^4.14.0
- firebase_auth ^6.6.1

Media & UI

- video_player ^2.9.2
- flutter_svg ^2.0.17

Dev Tools

- flutter_launcher_icons ^0.14.3

Architecture:

- lib
 - Controller
 - AuthController.dart
 - GenreController.dart
 - MovieController.dart
 - MovieDetailsController.dart
 - SearchController.dart
 - Database
 - DatabaseServices.dart
 - SharedPService.dart
 - Model
 - CastModel.dart
 - GenreModel.dart
 - MovieModel.dart

- Provider
 - FirebaseAuthProvider.dart
 - GenreProvider.dart
 - MovieDetailsProvider.dart
 - MovieProvider.dart
 - SearchProvider.dart
 - WatchlistProvider.dart
- Service
 - firebase_options.dart
 - FirebaseAuthService.dart
 - TMDBAPIService.dart

- Utils
 - AppTheme.dart
- Views
 - FavoritesScreen.dart
 - GenreMoviesScreen.dart
 - Homepage.dart
 - LoginScreen.dart
 - MovieDetailsScreen.dart
 - ProfileScreen.dart
 - SearchByGenreScreen.dart
 - SearchResultsScreen.dart
 - SignupScreen.dart
 - SplashScreen.dart
 - ToWatchScreen.dart
 - WatchedScreen.dart
 - WatchingScreen.dart
 - WatchListScreen.dart

- Widgets
 - ActionButtonWidget.dart
 - CastCardWidget.dart
 - EmptyStateWidget.dart
 - ErrorRetryWidget.dart
 - InfoTileWidget.dart
 - ListCardWidget.dart
 - LoadingWidget.dart
 - MovieCard.dart
 - MovieGridView.dart
 - MovieSection.dart
 - PaginationBar.dart
 - main.dart

Data Flow:

View > Controller > Provider > Service/Database > Model > Provider > View

Installation Guide:

- Note: you have to have flutter and Dart SDK

Steps:

```
bash

# 1. Clone the repository
git clone<your-repo-url>
cd iti_project
# 2. Install dependencies
flutter pub get
# 3. Run the app
flutter run
```

API setup Instructions:

- Making an account on TMDB and getting the API key.
- Code:

```
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../Model/MovieModel.dart';
import '../Model/GenreModel.dart';
import '../Model/CastModel.dart';

class TMDBApiService {
  final String apiKey = "API KEY HERE";

  List<MovieModel> _parseAndFilterMovies(List<dynamic> results) {
    return results
      .map((e) => MovieModel.fromJson(e))
      .where((movie) => !movie.adult && !movie.softcore)
      .toList();
  }

  Future<List<Genre>> getGenres() async {
    final url = Uri.parse("https://api.themoviedb.org/3/genre/movie/list?api_key=$apiKey");
    final response = await http.get(url);

    if (response.statusCode == 200) {
      final data = jsonDecode(response.body);
      final genreModel = GenreModel.fromJson(data);
      return genreModel.genres;
    } else {
      throw Exception("Failed to load genres: HTTP ${response.statusCode}");
    }
  }
}
```

```

Future<List<MovieModel>> getPopularMovies() async {
    final url =
Uri.parse("https://api.themoviedb.org/3/movie/popular?api_key=$apiKey&include_adult=false");
    final response = await http.get(url);

    if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        final List results = data["results"] ?? [];
        return _parseAndFilterMovies(results);
    } else {
        throw Exception("Failed to load popular movies");
    }
}

Future<List<MovieModel>> getTopRatedMovies() async {
    final url =
Uri.parse("https://api.themoviedb.org/3/discover/movie?api_key=$apiKey&sort_by=vote_average.desc&vote_count.gte=300&include_adult=false");
    final response = await http.get(url);

    if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        final List results = data["results"] ?? [];
        return _parseAndFilterMovies(results);
    } else {
        throw Exception("Failed to load top rated movies");
    }
}

Future<List<MovieModel>> getNowPlayingMovies() async {
    final url =
Uri.parse("https://api.themoviedb.org/3/movie/now_playing?api_key=$apiKey&include_adult=false");
    final response = await http.get(url);

    if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        final List results = data["results"] ?? [];
        return _parseAndFilterMovies(results);
    } else {
        throw Exception("Failed to load now playing movies");
    }
}

Future<List<MovieModel>> getUpcomingMovies() async {
    final url =
Uri.parse("https://api.themoviedb.org/3/movie/upcoming?api_key=$apiKey&region=US&include_adult=false");
    final response = await http.get(url);

    if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        final List results = data["results"] ?? [];
        return _parseAndFilterMovies(results);
    } else {
        throw Exception("Failed to load upcoming movies");
    }
}

Future<Map<String, dynamic>> searchMoviesPaginated(String query, {int page = 1}) async {
    final uri = Uri.https("api.themoviedb.org", "/3/search/movie", {
        "api_key": apiKey,
        "query": query.trim(),

```



```

        "page": page.toString(),
        "include_adult": "false",
    });
    final response = await http.get(uri);

    if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        final List results = data["results"] ?? [];
        final List<MovieModel> movies = _parseAndFilterMovies(results);
        return {
            "movies": movies,
            "page": (data["page"] as num?)?.toInt() ?? page,
            "totalPages": (data["total_pages"] as num?)?.toInt() ?? 1,
            "totalResults": (data["total_results"] as num?)?.toInt() ?? 0,
        };
    } else {
        throw Exception("Failed to search movies: HTTP ${response.statusCode}");
    }
}

Future<Map<String, dynamic>> getMoviesByGenre(int genreId, {int page = 1}) async {
    final uri = Uri.https("api.themoviedb.org", "/3/discover/movie", {
        "api_key": apiKey,
        "with_genres": genreId.toString(),
        "page": page.toString(),
        "include_adult": "false",
    });
    final response = await http.get(uri);

    if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        final List results = data["results"] ?? [];
        final List<MovieModel> movies = _parseAndFilterMovies(results);
        return {
            "movies": movies,
            "page": (data["page"] as num?)?.toInt() ?? page,
            "totalPages": (data["total_pages"] as num?)?.toInt() ?? 1,
            "totalResults": (data["total_results"] as num?)?.toInt() ?? 0,
        };
    } else {
        throw Exception("Failed to load genre movies: HTTP ${response.statusCode}");
    }
}

Future<CastModel> getMovieCredits(int movieId) async {
    final url = Uri.parse("https://api.themoviedb.org/3/movie/$movieId/credits?api_key=$apiKey");
    final response = await http.get(url);

    if (response.statusCode == 200) {
        final data = jsonDecode(response.body);
        return CastModel.fromJson(data);
    } else {
        throw Exception("Failed to load movie credits: HTTP ${response.statusCode}");
    }
}

Future<Map<String, dynamic>> getMovieDetailsFull(int movieId) async {
    final url =
Uri.parse("https://api.themoviedb.org/3/movie/$movieId?api_key=$apiKey&append_to_response=credits
");
    final response = await http.get(url);

```

```

    if (response.statusCode == 200) {
      return jsonDecode(response.body);
    } else {
      throw Exception("Failed to load full movie details: HTTP ${response.statusCode}");
    }
  }
}

```

Code details:

- `parseAndFilterMovies` —> Used to get movie lists with filtering. It depends that when any of the property (Adult or softcore) comes with true the result is false and the movie won't be added to the list. this function is set to private since we won't use it except in the file.
- Get functions (`GetGenres`, `getPopularMovies`, `getTopRated`, `getNowPlaying`) —> includes double filtering so that there is no room for error, using `include_adult = false` and `parseAndFilter`.
- Search functions —> it uses your input query to search in the database and returns the list of movies, total pages and total results.

FireBase:

- Firebase is used for authentication of user accounts.
 - logging in
 - sign up
 - logging out
 - changing password
- Code:

```

import 'package:firebase_auth/firebase_auth.dart';

class FirebaseAuthService {
  final FirebaseAuth _auth = FirebaseAuth.instance;

  Future<UserCredential> login({
    required String email,
    required String password,
  }) async {
    return await _auth.signInWithEmailAndPassword(
      email: email.trim(), password: password.trim());
  }

  Future<UserCredential> signUp({
    required String email,
    required String password,
  }) async {
    return await _auth.createUserWithEmailAndPassword(
      email: email.trim(),
      password: password.trim(),
    );
  }

  Future<void> logout() async {
    await _auth.signOut();
  }
}

```

```

Future<void> confirmCurrentPassword(String password) async {
  final user = _auth.currentUser;
  if (user == null || user.email == null) {
    throw FirebaseAuthException(
      code: 'no-current-user',
    );
  }
  final credential = EmailAuthProvider.credential(
    email: user.email!,
    password: password.trim(),
  );
  await user.reauthenticateWithCredential(credential);
}

Future<void> updatePassword(String newPassword) async {
  final user = _auth.currentUser;
  if (user == null) {
    throw FirebaseAuthException(
      code: 'no-current-user',
    );
  }
  await user.updatePassword(newPassword.trim());
}

User? get currentUser {
  return _auth.currentUser;
}
}

```

Code details:

- trim() is used to avoid trailing spaces giving us false negative when we log in or sign up.
- UserCredential: The object Firebase sends back containing details about the logged-in user and authentication token.
- ConfirmCurrentPassword is used for security importance. Imagine someone stole your device he will have access to change your password whenever he wants

DataBase:

- It handles Local storage
- Lists are dependent on User and Local storage so if you change accounts on same device your lists don't merge. and if you change devices or ports the lists are temporarily gone from the new device but will still be on the old device
- We use Shared preference for web and sqflite for mobile.
- we use if (kIsWeb) return null; because Web browsers don't have Sqflite file access so it changes to SharedPreferences
- Databases implement one instance so there is only one open database connection.

Application Flow:

1. Main() runs

2. Provider start up
3. SplashScreen Loads
4. Checks if User is logged in or not
5. LoginScreen if user isn't logged in
6. homepage: List of movies
7. from the homepage we can navigate to: SearchScreen, Profilescreen, Watlists screen, favorites screen, Genres screem
8. from any movie poster we can navigate to its detail.

Known Limitations:

- No Cloudsync for Movielists.
- Database for web don't share across differant ports
- Password field can sometimes freeze on web if there was a lot of mistakes and requires refreshing of the page.
- no trialer present in movie details.
- no strict content filtering
- no age verification
- no changing of font or language
- no user reviews
- no image caching which can cause a lot of internet usage.

AppTheme:

- Used Red and Gold for a theatre vibe and watch Lists and genres buttons took different bright colors that fit their function

```
import 'package:flutter/material.dart';

class AppTheme {
  static final ThemeData lightMode = ThemeData(
    brightness: Brightness.light,
    scaffoldBackgroundColor: Color.fromARGB(255, 250, 250, 249),
    colorScheme: ColorScheme.light(
      primary: Color.fromARGB(255, 218, 0, 0),
      onPrimary: Color.fromARGB(255, 28, 25, 23),
      secondary: Color.fromARGB(255, 219, 182, 58),
      onSecondary: Color.fromARGB(255, 28, 25, 23),
      tertiary: Color.fromARGB(255, 211, 211, 209),
      surface: Color.fromARGB(255, 250, 246, 240),
    ),
    appBarTheme: AppBarTheme(
      backgroundColor: Color.fromARGB(255, 250, 250, 249),
      foregroundColor: Color.fromARGB(255, 219, 182, 58),
      elevation: 0,
    ),
    elevatedButtonTheme: ElevatedButtonThemeData(
      style: ElevatedButton.styleFrom(
        backgroundColor: Color.fromARGB(255, 218, 0, 0),
        foregroundColor: Color.fromARGB(255, 250, 250, 249),
      ),
    ),
  );
}
```

```

    ),
  ),
  textTheme: TextTheme(
    titleMedium: TextStyle(
      color: Color.fromARGB(255, 28, 25, 23),
      fontWeight: FontWeight.bold,
    ),
    bodyMedium: TextStyle(color: Color(0xFF1C1917)),
  ),
);

static final ThemeData darkMode = ThemeData(
  brightness: Brightness.dark,
  scaffoldBackgroundColor: Color.fromARGB(255, 28, 25, 23),
  colorScheme: ColorScheme.dark(
    primary: Color.fromARGB(255, 218, 0, 0),
    onPrimary: Color.fromARGB(255, 28, 25, 23),
    secondary: Color.fromARGB(255, 219, 182, 58),
    onSecondary: Color.fromARGB(255, 28, 25, 23),
    tertiary: Color.fromARGB(255, 48, 48, 48),
    surface: Color.fromARGB(255, 56, 51, 50),
  ),
  appBarTheme: AppBarTheme(
    backgroundColor: Color.fromARGB(255, 28, 25, 23),
    foregroundColor: Color.fromARGB(255, 219, 182, 58),
    elevation: 0,
  ),
  elevatedButtonTheme: ElevatedButtonThemeData(
    style: ElevatedButton.styleFrom(
      backgroundColor: Color.fromARGB(255, 218, 0, 0),
      foregroundColor: Color.fromARGB(255, 28, 25, 23),
    ),
  ),
  textTheme: TextTheme(
    titleMedium: TextStyle(
      color: Color.fromARGB(255, 250, 250, 249),
      fontWeight: FontWeight.bold,
    ),
    bodyMedium: TextStyle(color: Color.fromARGB(255, 250, 250, 249)),
  ),
);
}

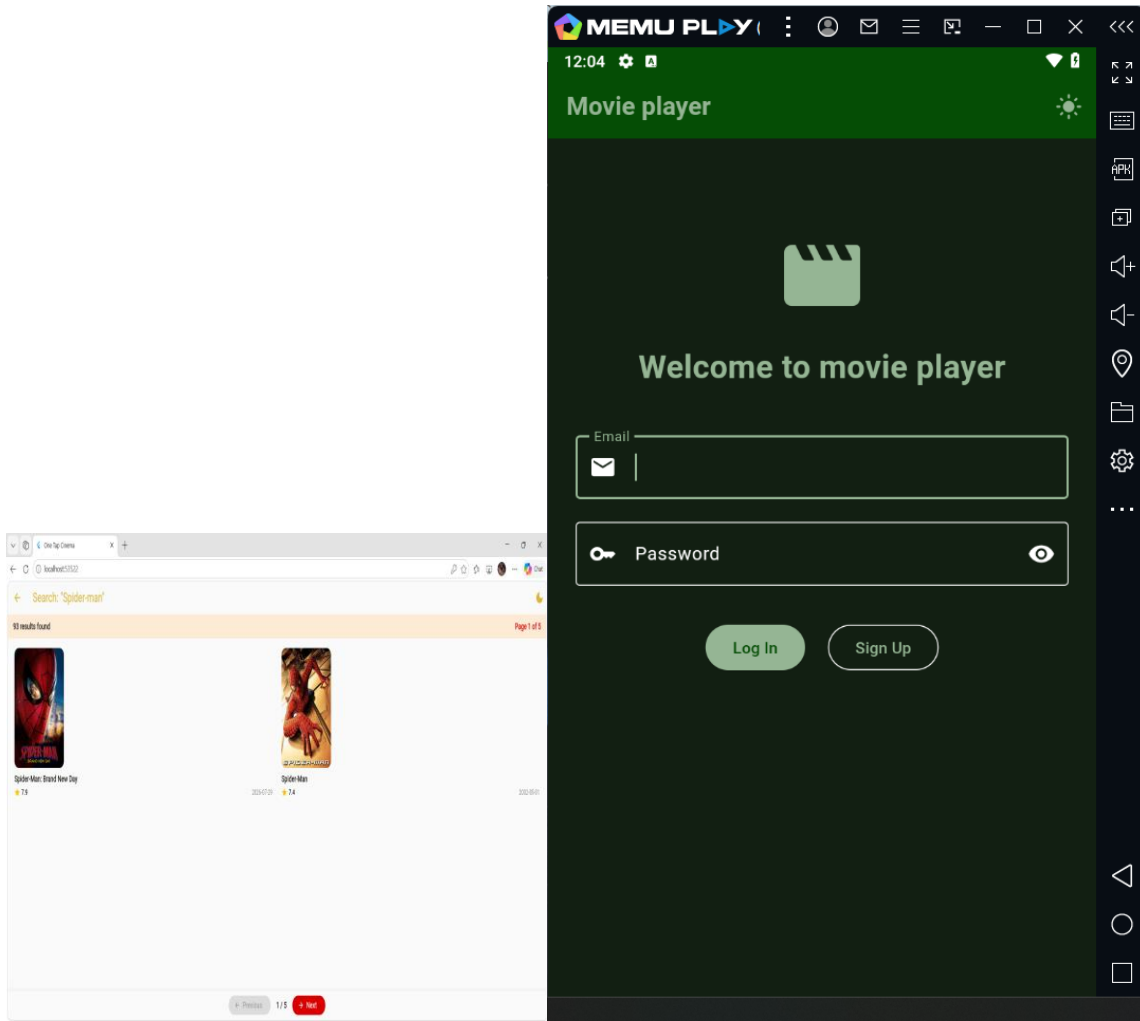
```

Check Notion file For Editing history:

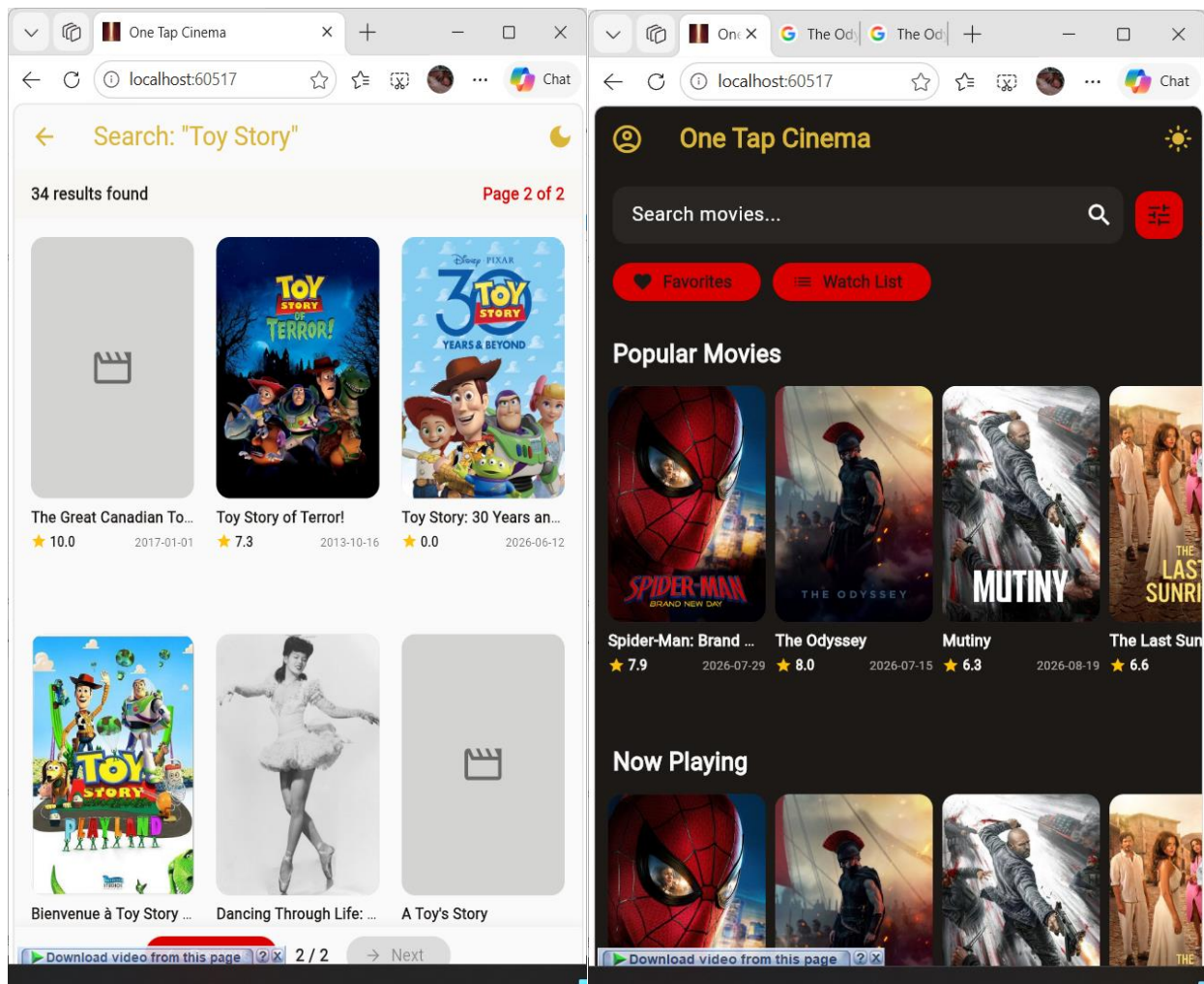
https://app.notion.com/p/ITI-final-project-3cab730d537f80ccb516c6ce25007c8b?source=copy_link

ScreenShots:

Old Screenshots:



New and Final Screen Shots:



One Tap Cinema

localhost:60517

Chat



The Odyssey

★ 8.0 / 10 (3401) 2h 53m EN

Favorite

Watching

To Watch

Watched

'Defy the gods.'

Adventure

Action

Fantasy

Storyline

Odysseus, the legendary King of Ithaca, embarks on a long and perilous journey home following the Trojan War. Throughout his voyage, he is forced to confront the whims of gods, mythological monsters, and trials that stretch both his cunning and his humanity to the breaking point.

Top Cast


non
us


Tom Holland
Telemachus


Anne Hathaway
Penelope


Robert Pattinson
Antinous


Himesh Patel
Eurylochus

Directed by: **Christopher Nolan**

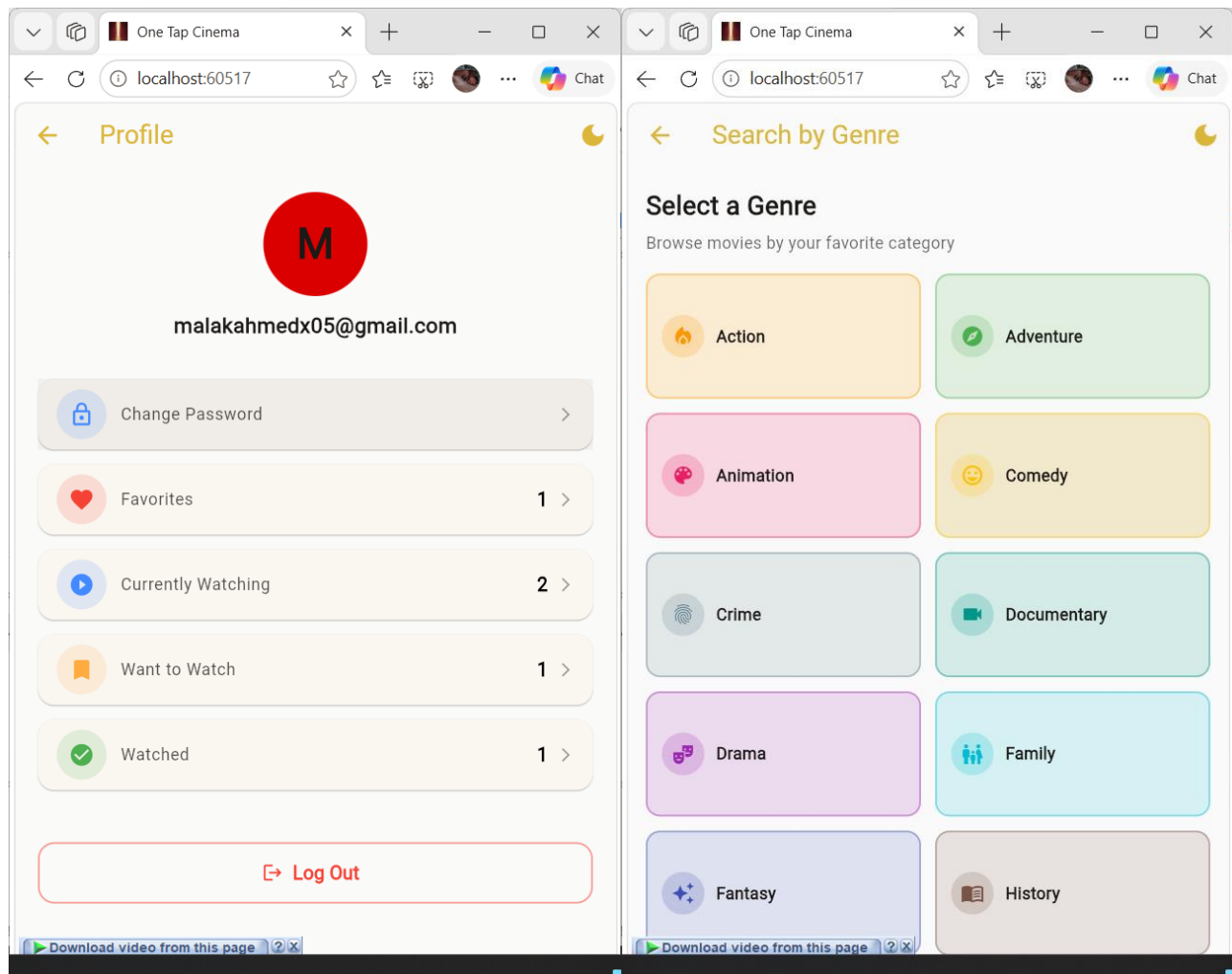
Information

Release Date	2026-07-15
Status	Released
Budget	\$250,000,000
Revenue	\$1,555,182,696

Trigger Warning

Parent Guide

Download video from this page



One Tap Cinema

localhost:60517

Comedy Movies

174924 movies

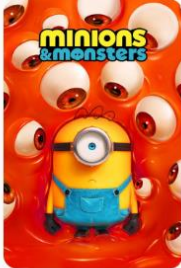
Page 1 of 8747



Coyote vs. Acme
★ 7.8 2026-08-20



Toy Story 5
★ 8.3 2026-06-17



Minions & Monsters
★ 7.6 2026-06-24



Moana



Ghost in the Cell



Rosebush Pruning

Download video from this page

1 / 8747

Next

One Tap Cinema

localhost:60517

Search: "Toy Story"

34 results found

Page 1 of 2



Toy Story 5
★ 8.3 2026-06-17



Toy Story
★ 8.0 1995-11-22



Toy Story 4
★ 7.5 2019-06-19



Toy Story 2



Toy Story 3



Toy Story That Time F...

Download video from this page

1 / 2

Next